

Curso online: **Programación orientada a objetos: Java avanzado**

Conexión a Bases de Datos

Autores

Jorge Molinero Muñoz

Miguel Sallent Sánchez

ÍNDICE

1. Creación de la base de datos.....	3
2. JDBC.....	7
3. Configuración de Eclipse.....	7
4. Conexión a la base de datos desde Eclipse.....	12
5. Modificación de valores en base de datos.....	14
6. Prepared Statements.....	16
7. Ejercicio resuelto.....	19
8. Recursos.....	19

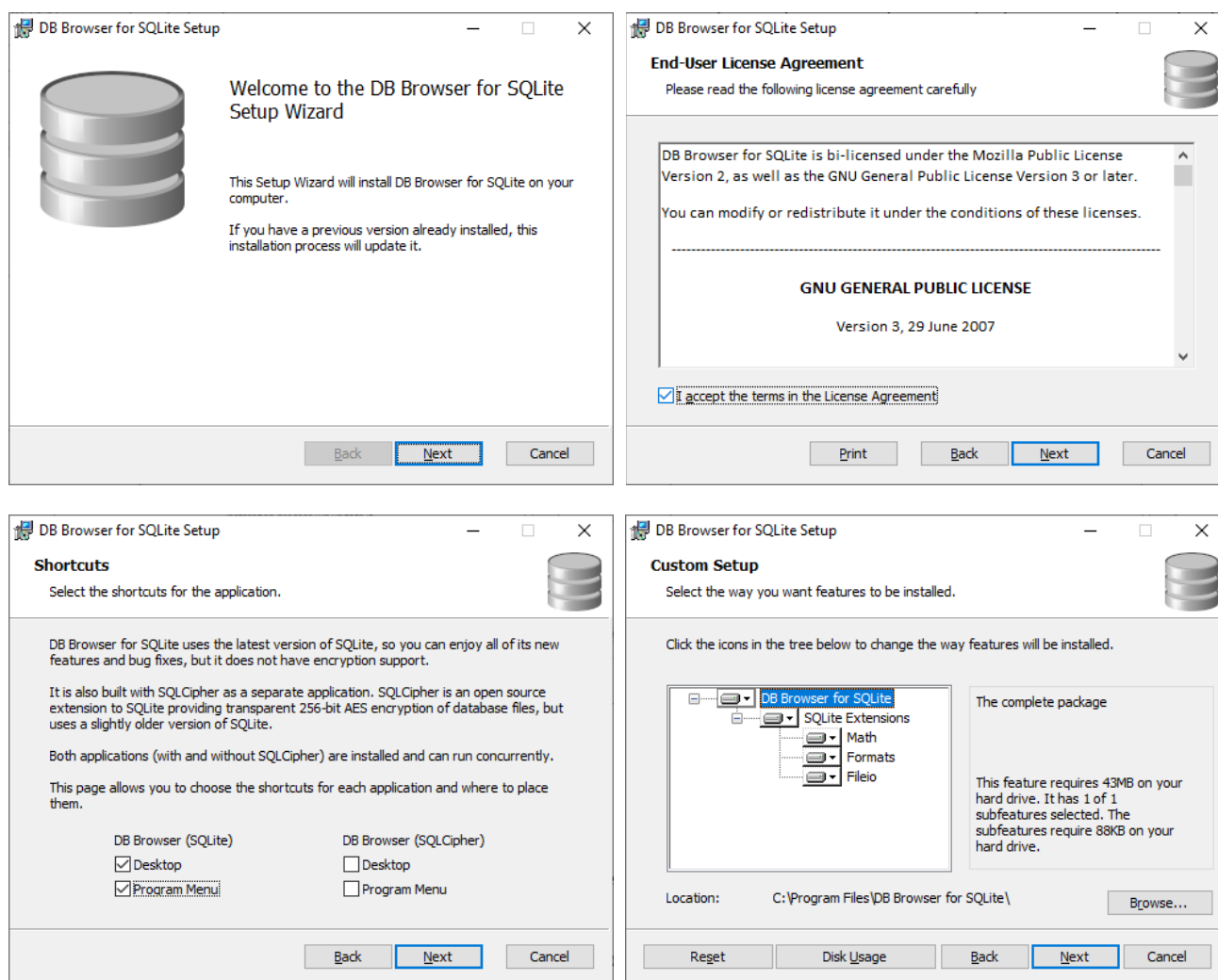
1. Creación de la base de datos

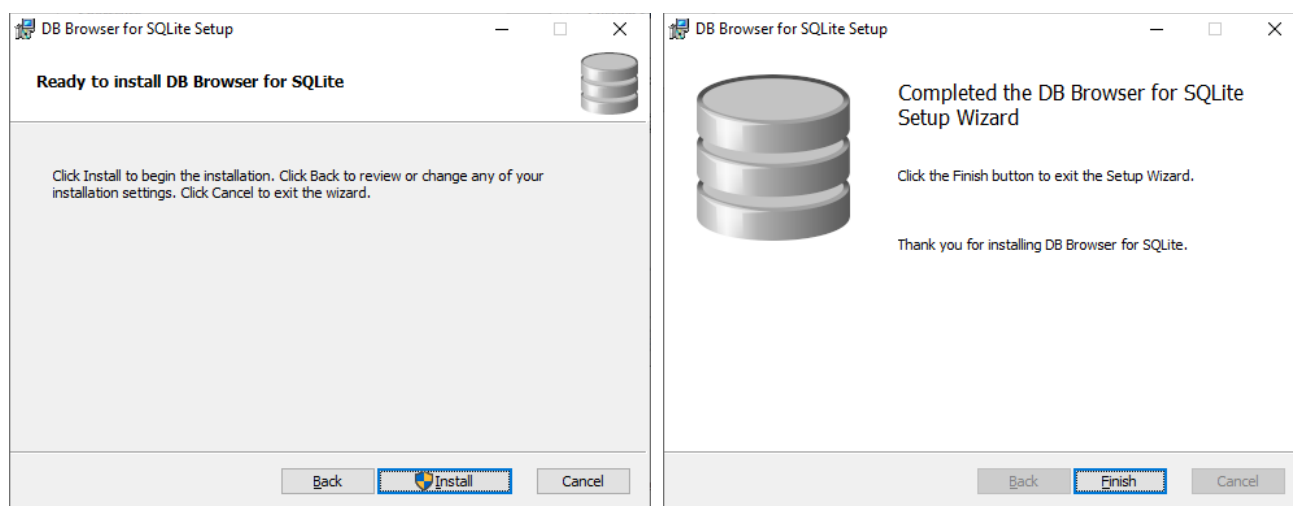
Para este curso vamos a trabajar con el motor de base de datos SQLite. La ventaja principal es que es un sistema muy ligero, que no sigue una filosofía cliente-servidor sino que toda la información de una base de datos se almacena en un fichero al que los clientes pueden acceder directamente. Además es de dominio público y gratuita.

No obstante, vamos a instalar la herramienta DB Browser para SQLite, que es una herramienta de código abierto que permite crear, diseñar y editar archivos de base de datos compatibles con SQLite. Esto nos va a permitir comprobar fácilmente que los cambios que realicemos en la base de datos desde nuestras aplicaciones Java se efectúan correctamente. La descarga gratuita se puede realizar en el siguiente enlace:

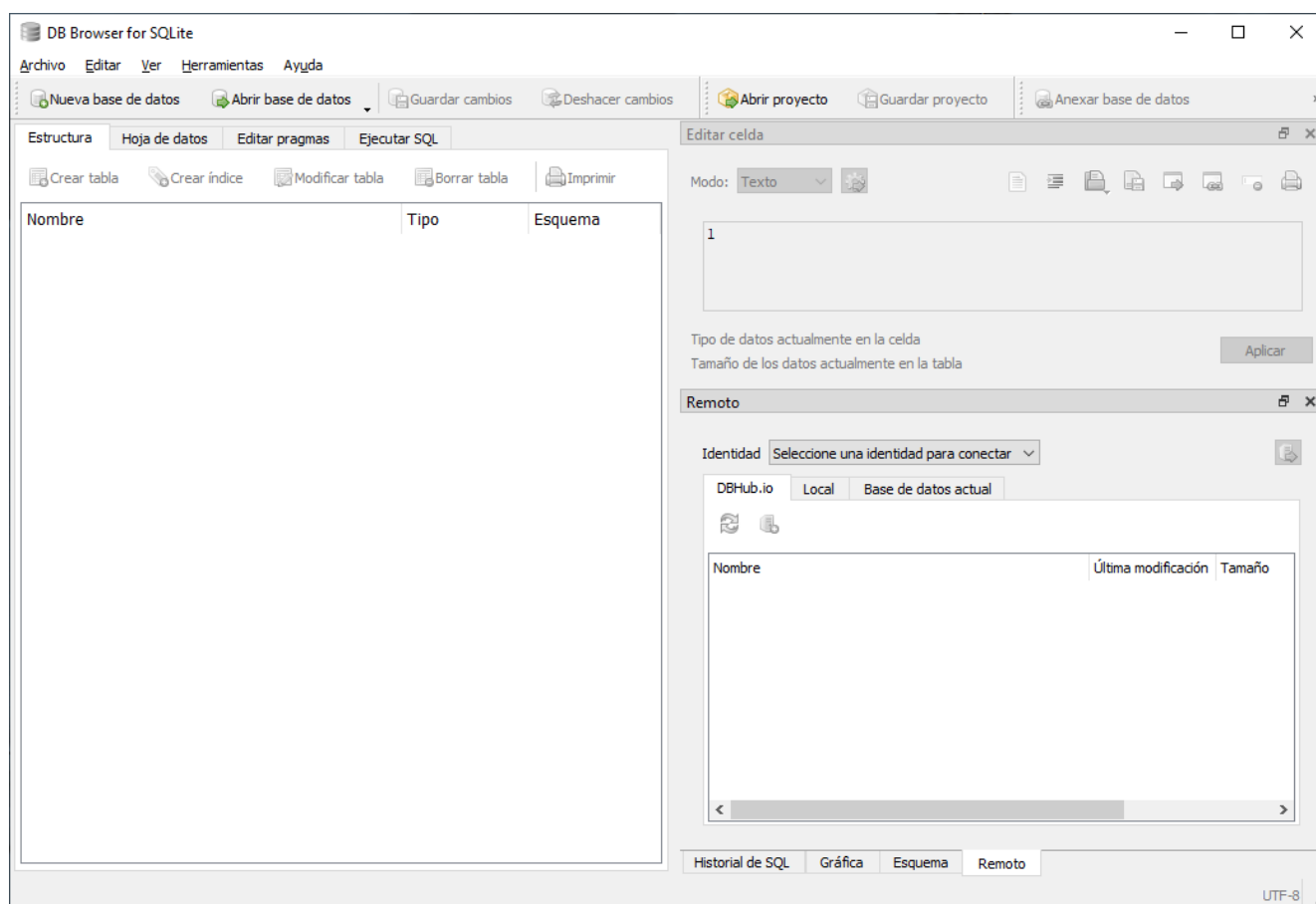
<https://sqlitebrowser.org/dl/>

En esta web debemos seleccionar y descargar el instalador que se ajusta a nuestro equipo (Windows 32 ó 64 bits, Mac o Linux). A continuación, se muestran los pasos para la instalación en un equipo Windows 64 bits. Dejaremos las opciones de instalación por defecto:

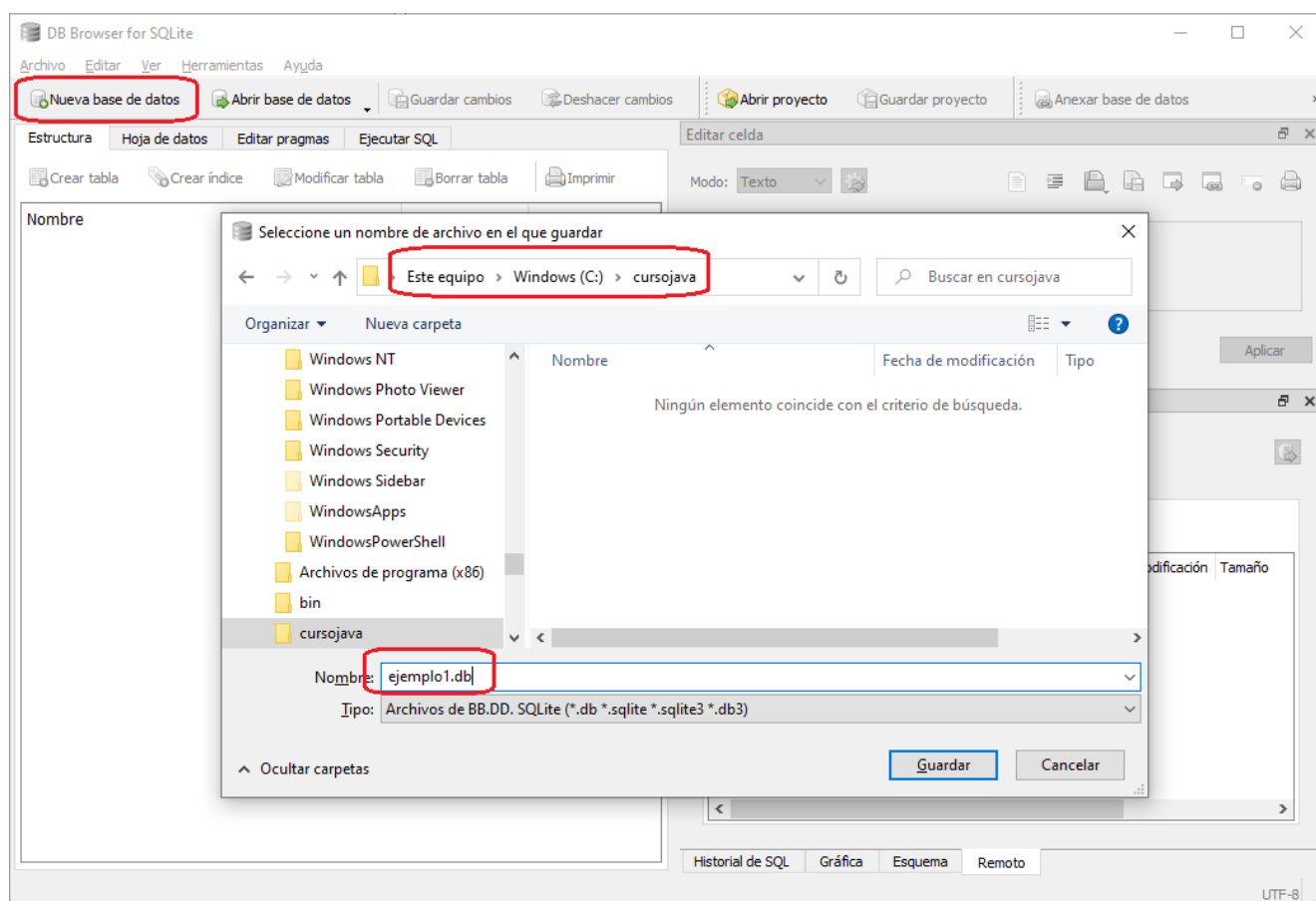




Una vez finalizada la instalación ejecutaremos la aplicación:

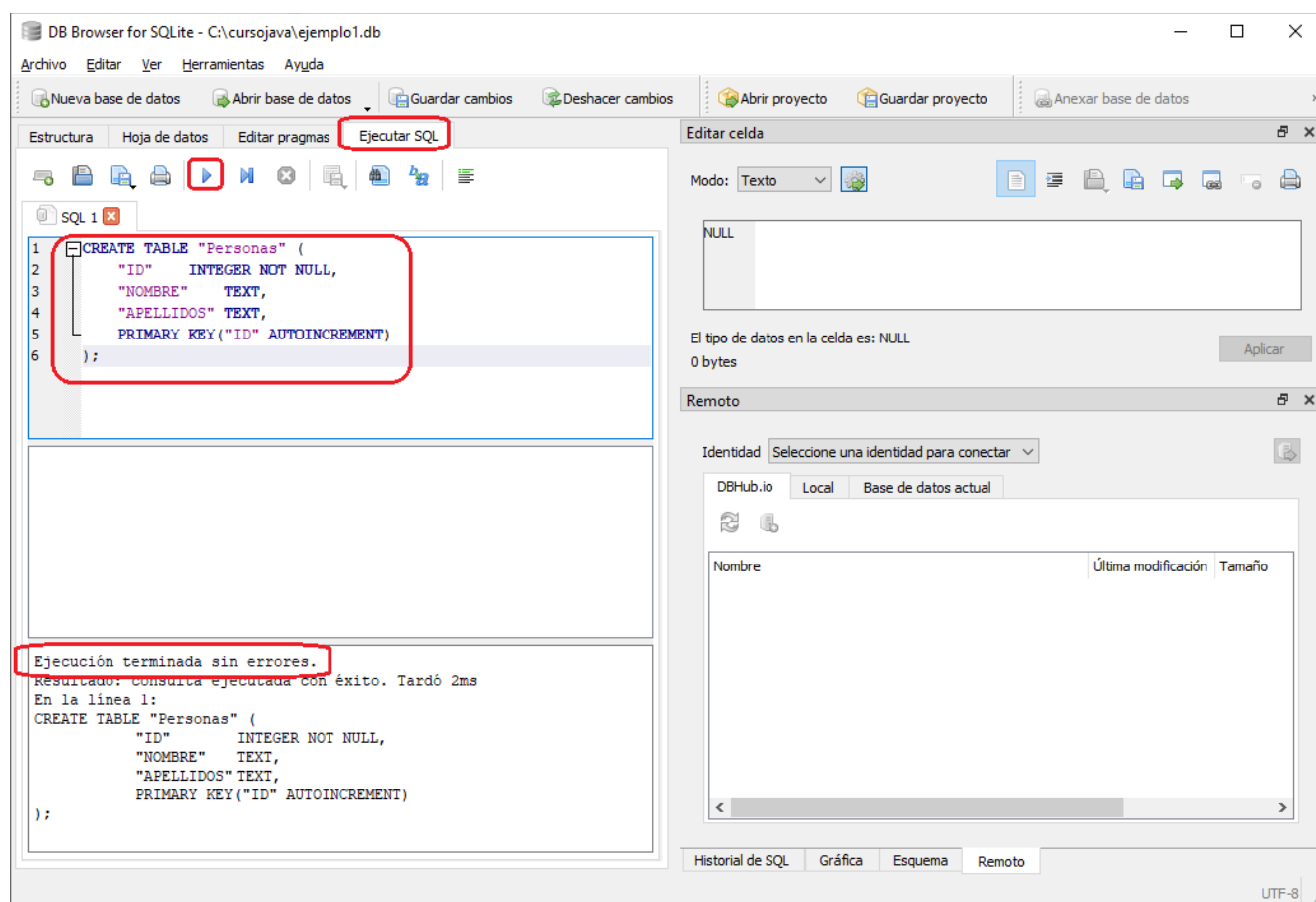


A continuación vamos a crear una base de datos para realizar las primeras pruebas. Para ello utilizaremos el botón “Nueva base de datos”, y nos pedirá que indiquemos una ubicación y un nombre para el fichero que contendrá la base datos. Por ejemplo:

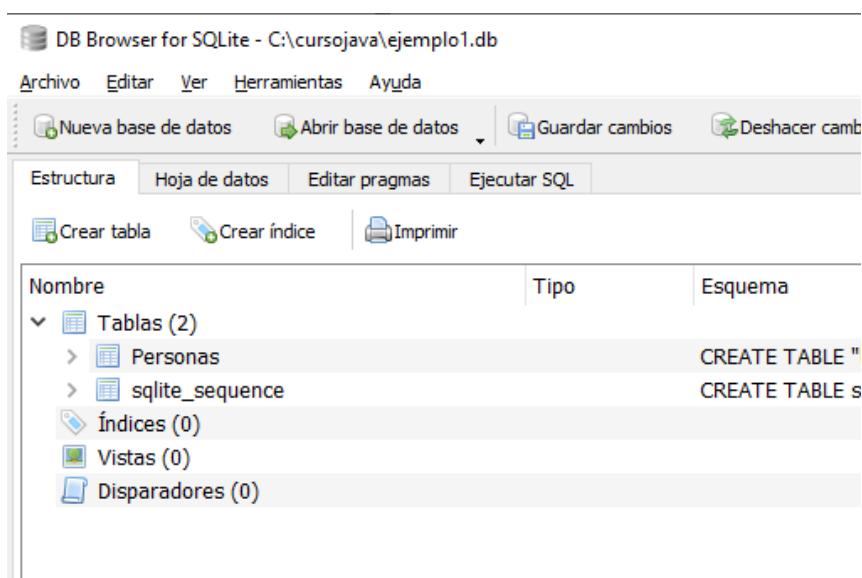


A continuación nos aparecerá una ventana para agregar una nueva tabla. En esta ventana podemos añadir los campos que tendrá la tabla. Otra opción es ejecutar un script sql para crear la tabla, esta será la opción que utilizaremos en este ejemplo. Para ello, cerraremos la ventana de definición de nueva tabla, seleccionaremos la pestaña Ejecutar SQL y ejecutaremos el siguiente script:

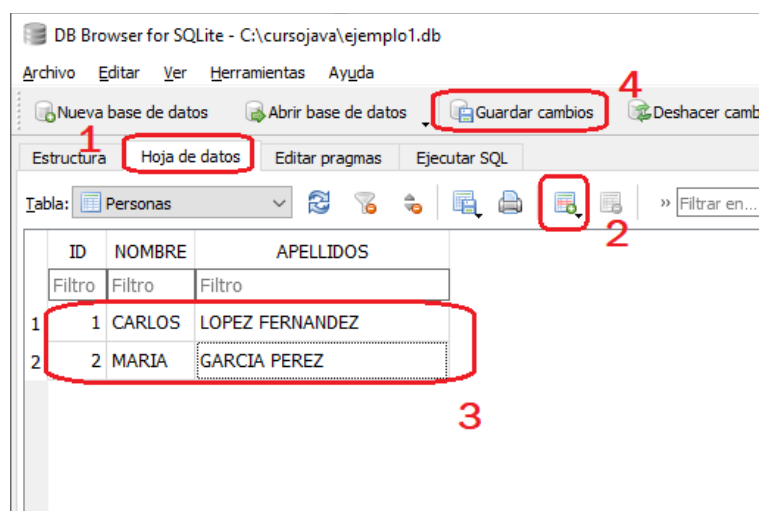
```
CREATE TABLE "Personas" (
    "ID"    INTEGER NOT NULL,
    "NOMBRE" TEXT,
    "APELLIDOS" TEXT,
    PRIMARY KEY("ID" AUTOINCREMENT)
);
```



Si cambiamos a la pestaña Estructura veremos la tabla creada:



El siguiente paso es insertar algunas filas en la tabla para realizar las pruebas. Para ello utilizaremos la pestaña Hoja de datos, en la que hay que pulsar el botón "insertar nuevo registro". Puesto que hemos definido el campo ID como autoincrementable únicamente será necesario rellenar los valores de nombre y apellidos:



Es necesario pulsar el botón Guardar Cambios para que los cambios se trasladen al fichero de base de datos.

En los siguientes apartados veremos cómo conectarnos a la base de datos desde Java y recuperar las filas que hemos insertado.

2. JDBC

Para conectar a base de datos vamos a usar JDBC (Java Database Connectivity), que es el API de Java para acceder a base de datos. La arquitectura consta de dos elementos:

- El API JDBC: es el conjunto de clases que Java ofrece para acceder a bases de datos, y que es independiente del gestor de base de datos utilizado. Es parte de la plataforma de Java y viene incluido tanto en Java SE como en Java EE, en los paquetes `java.sql` y `javax.sql`. Permite gestionar conexiones, consultas, resultados, etc.
- El driver de la base de datos: es el conjunto de clases que permite acceder a un tipo de base de datos en concreto (SQL Server, SQLite, Oracle, etc). En función de la base de datos utilizada, tendremos que utilizar un driver distinto. Los drivers son suministrados por los fabricantes de las bases de datos, no por Java. Cualquier base de datos puede implementar un driver, siempre que se ajuste a las especificaciones de jdbc. Los drivers se suelen suministrar como archivos jar.

Esta arquitectura por tanto separa “el interfaz”, que es el API JDBC, de “la implementación”, que es el driver. Una aplicación podría cambiar de motor de base de datos simplemente cambiando el driver y realizando unos cambios mínimos en el código.

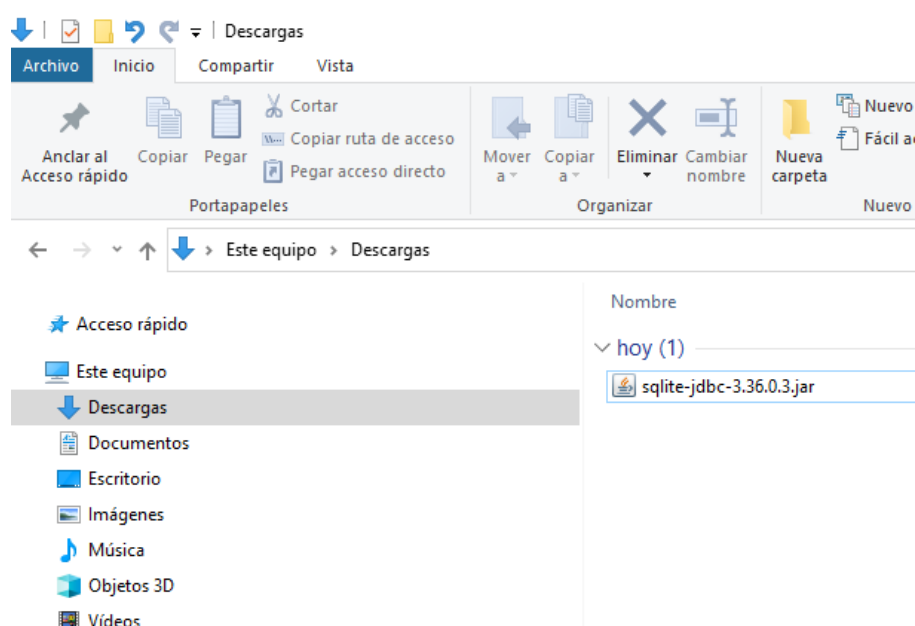
3. Configuración de Eclipse

Como se ha indicado, los drivers no están incluidos en la distribución de Java, por tanto tampoco están en Eclipse. Lo primero que deberemos hacer es configurar Eclipse para que utilice el driver adecuado.

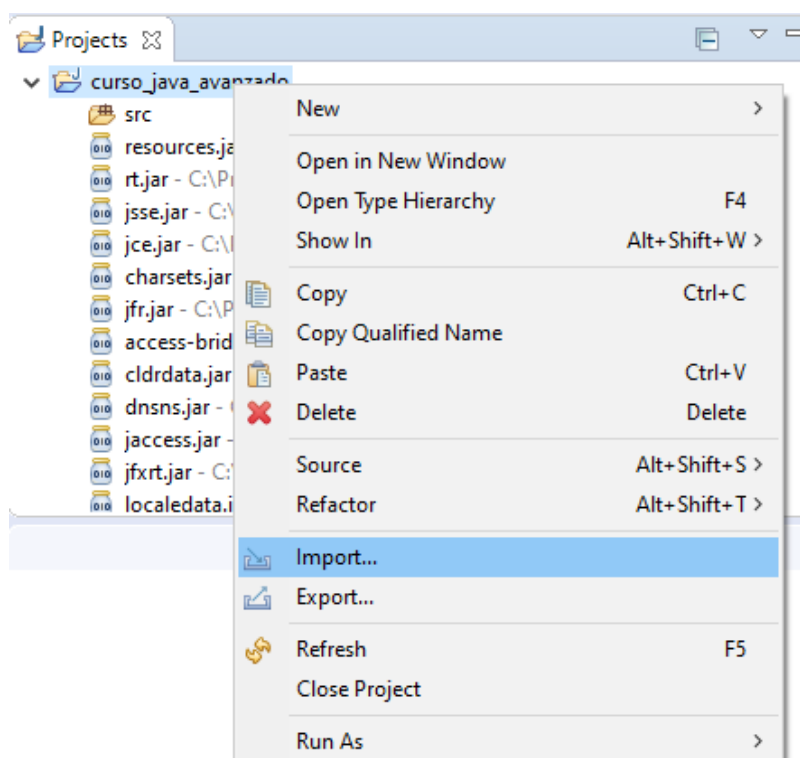
En primer lugar procederemos a descargar el driver jdbc para SQLite. Está disponible en la plataforma del INAP y también podemos descargarlo de internet:

<https://mvnrepository.com/artifact/org.xerial/sqlite-jdbc>

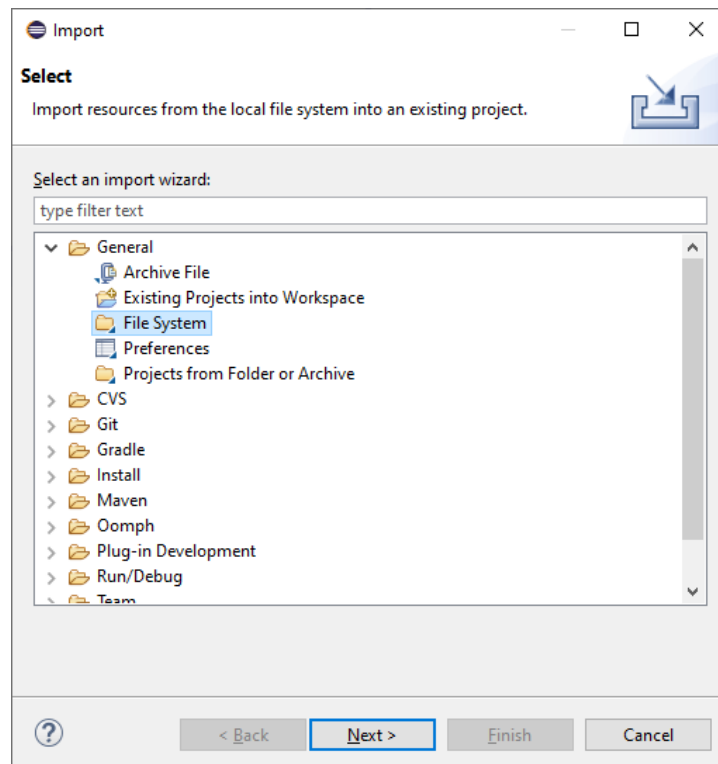
Descargaremos el fichero jar a una carpeta de nuestro ordenador, por ejemplo la carpeta Descargas:



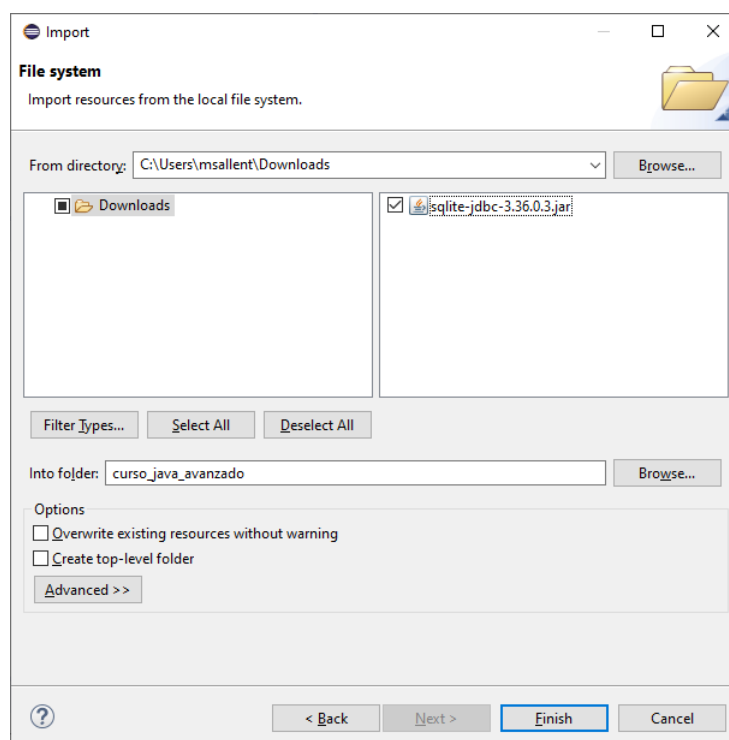
A continuación, en Eclipse pulsaremos con el botón derecho del ratón sobre el proyecto, y seleccionaremos la opción “Import...”:



Seleccionamos General -> File System:

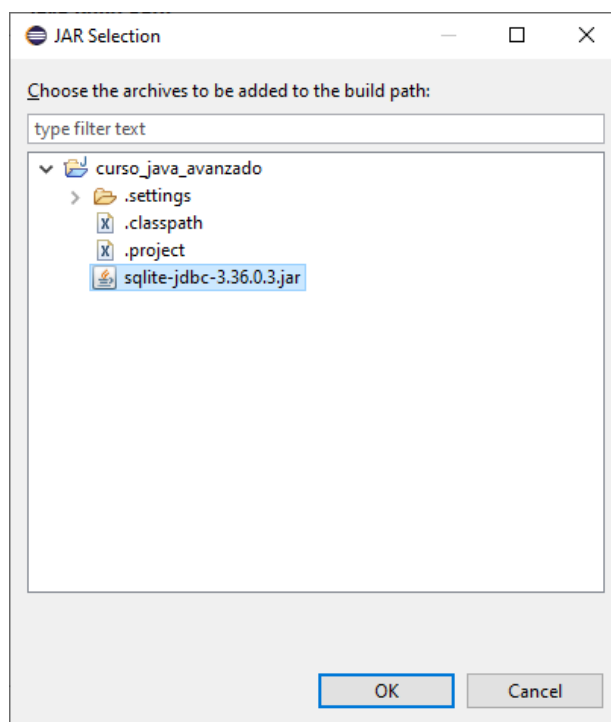
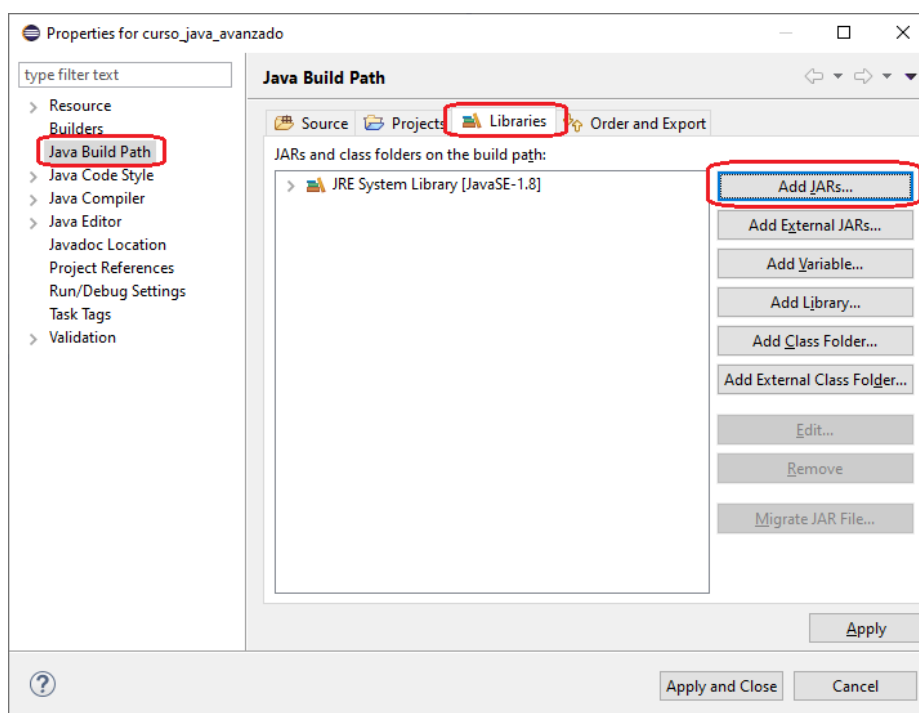


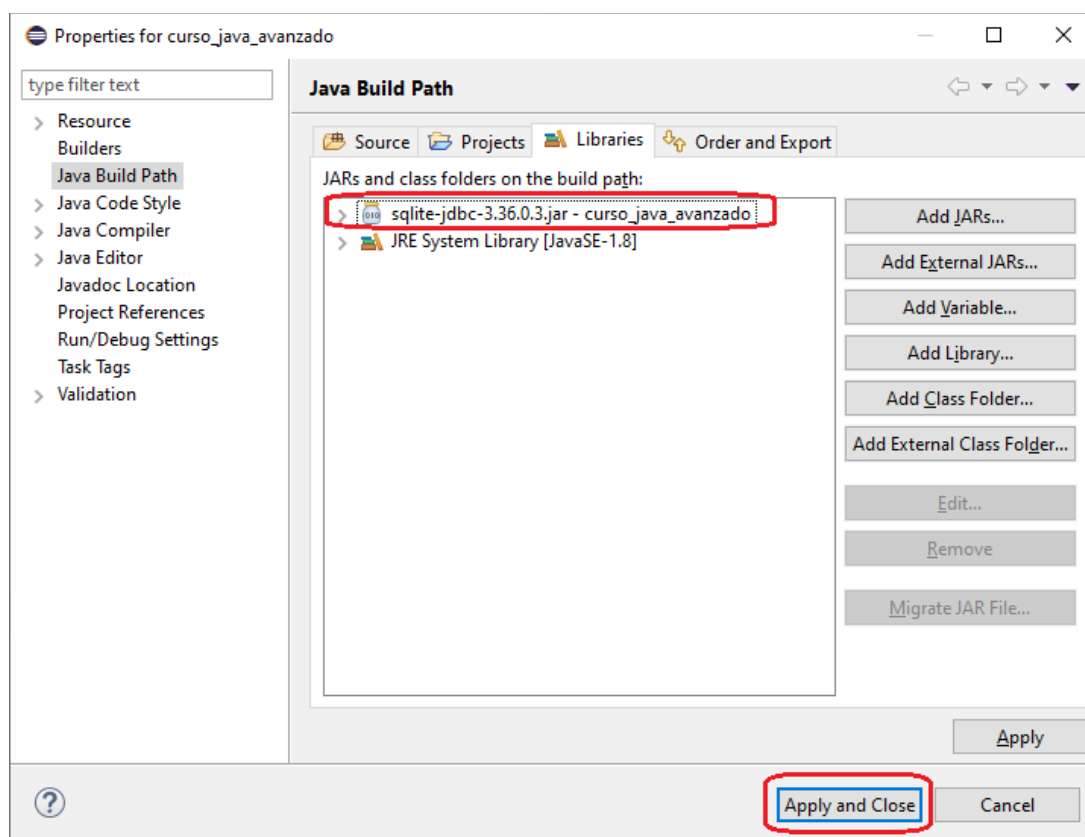
Buscamos la carpeta donde hemos descargado el driver y lo seleccionamos:



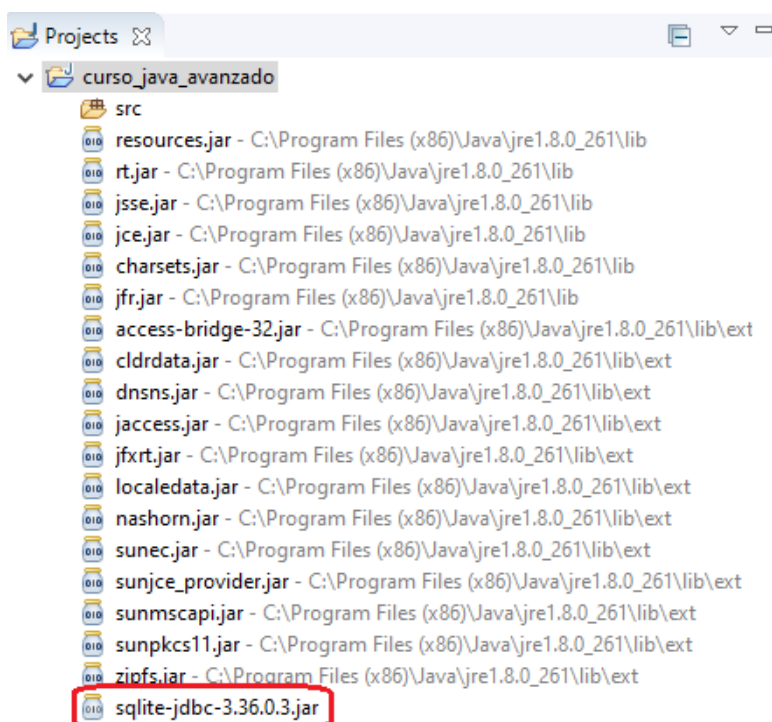
Una vez hecho esto tendremos una copia del driver incorporada a los archivos de nuestro proyecto.

El siguiente paso es añadir el driver al conjunto de clases que utiliza el proyecto. Para ello con el botón derecho del ratón pulsamos sobre el proyecto y seleccionamos "Properties". En la ventana que aparece añadimos el driver al Java Build Path tal y como se muestra a continuación:





Una vez realizados todos los pasos, desde la perspectiva Java Browsing de Eclipse podemos comprobar que el driver se ha añadido correctamente al proyecto:



4. Conexión a la base de datos desde Eclipse

Una vez que hemos configurado Eclipse podemos crear nuestra primera clase que se conecte a la base de datos.

En este ejemplo simplemente nos conectamos a base de datos y mostramos todas las filas de la tabla Personas. Antes de ejecutar el programa hay que editar la ruta donde se encuentra la base de datos:

```
package tema2;

import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

import org.sqlite.SQLiteDataSource;

public class SelectExample {
    public static void main(String[] args) {

        SQLiteDataSource ds = null;

        Connection conn = null;
        Statement stmt = null;

        try {
            ds = new SQLiteDataSource();

            ds.setUrl("jdbc:sqlite:C:/cursojava/ejemplo1.db");

            String query = "SELECT * FROM Personas";

            conn = ds.getConnection();
            stmt = conn.createStatement();

            ResultSet rs = stmt.executeQuery(query);

            while (rs.next()) {
                int id = rs.getInt("ID");
                String nombre = rs.getString("NOMBRE");
                String apellidos = rs.getString("APELLIDOS");

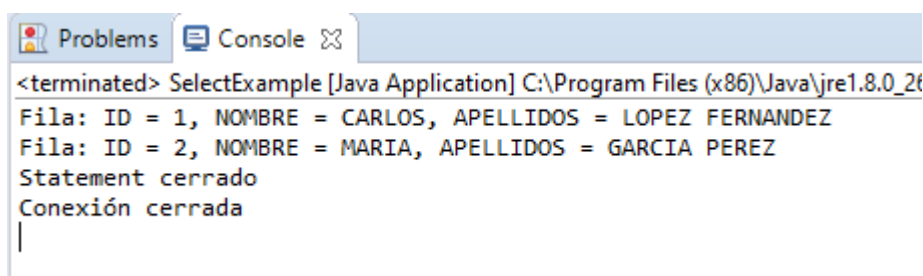
                System.out.println("Fila: ID = " + id + ", NOMBRE = " + nombre +
                    ", APELLIDOS = " + apellidos);
            }
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
            e.printStackTrace(System.err);
        } finally {
            if (stmt != null) {
                try {
                    stmt.close();
                    System.out.println("Statement cerrado");
                } catch (SQLException e) {
                    System.err.println("Error al cerrar Statement");
                    e.printStackTrace();
                }
            }
        }
    }
}
```

```

    }
    if (conn != null) {
        try {
            conn.close();
            System.out.println("Conexión cerrada");
        } catch (SQLException e) {
            System.err.println("Error al cerrar la conexión");
            e.printStackTrace();
        }
    }
}
}
}
}
}

```

En el ejemplo hemos ejecutado la consulta SQL “SELECT * FROM Personas”, que quiere decir “Seleccionar todos los campos de la tabla Personas, sin ningún filtro para las filas”. El resultado será algo así:



```

<terminated> SelectExample [Java Application] C:\Program Files (x86)\Java\jre1.8.0_2t
Fila: ID = 1, NOMBRE = CARLOS, APELLIDOS = LOPEZ FERNANDEZ
Fila: ID = 2, NOMBRE = MARIA, APELLIDOS = GARCIA PEREZ
Statement cerrado
Conexión cerrada

```

Obsérvese que el cierre de los objetos de base de datos se ha incluido en una cláusula *finally*. Esto es muy importante, ya que la conexión debe cerrarse tanto si hay errores como si no. Si quedan conexiones sin cerrar se puede alcanzar el límite de conexiones abiertas que tenga establecido el motor de base de datos y a partir de ese momento fallarían todas las consultas.

Alternativamente podemos usar una cláusula *try-with-resources*, especificando entre paréntesis los objetos que queremos que se cierren al finalizar el bloque *try*. Esto ocurrirá tanto si se lanzan excepciones como si no, por tanto equivale al bloque *finally*. El código podría quedar de la siguiente manera:

```

package tema2;

import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

import org.sqlite.SQLiteDataSource;

public class SelectExample2 {
    public static void main(String[] args) {

        SQLiteDataSource ds = new SQLiteDataSource();
        ds.setUrl("jdbc:sqlite:C:/cursojava/ejemplo1.db");

        String query = "SELECT * FROM Personas";

        try (Connection conn = ds.getConnection());

```

```

Statement stmt = conn.createStatement(); {

    ResultSet rs = stmt.executeQuery(query);

    while (rs.next()) {
        int id = rs.getInt("ID");
        String nombre = rs.getString("NOMBRE");
        String apellidos = rs.getString("APELLIDOS");

        System.out.println("Fila: ID = " + id + ", NOMBRE = " + nombre +
            ", APELLIDOS = " + apellidos);
    }
} catch (SQLException e) {
    System.err.println("Error: " + e.getMessage());
    e.printStackTrace(System.err);
}
}

```

Como vemos esta forma es más compacta y además menos propensa a errores, por lo que es la recomendada.

No es objeto del curso profundizar en la sintaxis SQL, ya que es compleja y daría para un curso completo. El lenguaje SQL es un estándar independiente del motor de base de datos utilizado. Mostramos aquí algunos ejemplos básicos:

CONSULTA	SIGNIFICADO
SELECT * FROM PERSONAS	Seleccionar todos los campos de todas las filas de la tabla Personas
SELECT ID, NOMBRE FROM PERSONAS	Seleccionar los campos ID y NOMBRE de la tabla Personas (todas las filas)
SELECT * FROM PERSONAS WHERE NOMBRE = 'MARIA'	Seleccionar todos los campos de la tabla Persona, sólo las filas cuyo campo NOMBRE tome el valor MARIA.
SELECT * FROM PERSONAS WHERE NOMBRE = 'MARIA' AND APELLIDOS = 'GARCIA'	Seleccionar todos los campos de la tabla Persona, sólo las filas cuyo campo NOMBRE tome el valor MARIA y cuyo campo APELLIDOS tome el valor GARCIA
SELECT * FROM PERSONAS WHERE NOMBRE = 'MARIA' OR APELLIDOS = 'GARCIA'	Seleccionar todos los campos de la tabla Persona, sólo las filas cuyo campo NOMBRE tome el valor MARIA o cuyo campo APELLIDOS tome el valor GARCIA

5. Modificación de valores en base de datos

En SQL hay tres tipos de operaciones de modificación:

- INSERT: inserción de filas
- DELETE: borrado de filas
- UPDATE: actualización de filas

Por ejemplo:

CONSULTA	SIGNIFICADO
INSERT INTO PERSONAS (NOMBRE, APELLIDOS) VALUES ('MARIA', 'GARCIA')	Inserta una fila, asignando valores a los campos NOMBRE y APELLIDOS.
DELETE FROM PERSONAS WHERE NOMBRE = 'MARIA'	Borra todas las filas cuyo campo NOMBRE toma el valor MARIA.
UPDATE PERSONAS SET NOMBRE = 'TERESA' WHERE ID = 1	Actualiza las filas cuyo campo ID toma el valor 1, estableciendo el campo NOMBRE con el valor TERESA.

Obsérvese que es necesario entrecomillar las cadenas de texto.

Para ejecutar sentencias de actualización, la sintaxis es muy similar a la de SELECT, pero en vez de utilizar el método `executeQuery` utilizaremos el método `executeUpdate`.

Por ejemplo, para insertar una fila el código sería el siguiente:

```
public static void main(String[] args) {

    SQLiteDataSource ds = new SQLiteDataSource();
    ds.setUrl("jdbc:sqlite:C:/cursojava/ejemplo1.db");

    try (Connection conn = ds.getConnection();
        Statement stmt = conn.createStatement();) {

        stmt.executeUpdate("INSERT INTO PERSONAS (NOMBRE, APELLIDOS) VALUES
        ('JUAN', 'MARTINEZ')");

    } catch (SQLException e) {
        System.err.println("Error: " + e.getMessage());
        e.printStackTrace(System.err);
    }
}
```

Comprobamos en el DB Browser que la fila se ha insertado (pulsar el botón Refrescar):

Estructura	Hoja de datos	Editar pragmas	Ejecutar SQL
------------	---------------	----------------	--------------

Tabla:	Personas				
--------	----------	---	---	--	---

	ID	NOMBRE	APELLIDOS
	Filtro	Filtro	Filtro
1	1	CARLOS	LOPEZ FERNANDEZ
2	2	MARIA	GARCIA PEREZ
3	3	JUAN	MARTINEZ

Se propone al alumno la ejecución de las siguientes sentencias para comprobar su resultado.

Modificación de fila:

```
stmt.executeUpdate("UPDATE PERSONAS SET NOMBRE = 'PEDRO' WHERE NOMBRE = 'JUAN'");
```

Borrado de fila:

```
stmt.executeUpdate("DELETE FROM PERSONAS WHERE NOMBRE = 'PEDRO'");
```

6. Prepared Statements

A veces es más conveniente usar un objeto PreparedStatement para enviar sentencias SQL a la base de datos, en lugar de un objeto Statement. Si se desea ejecutar una sentencia muchas veces, generalmente se reduce el tiempo de ejecución si se usa un objeto PreparedStatement.

La característica principal de un objeto PreparedStatement es que, a diferencia de un objeto Statement, recibe una sentencia SQL **cuando se crea**, no cuando se ejecuta. La ventaja de esto es que, en la mayoría de los casos, esta instrucción SQL se envía al motor de base de datos de inmediato, donde se precompila. Cuando se ejecuta la consulta ya está precompilada. Si ejecuto la consulta muchas veces, sólo habré precompilado una vez, ahorrando tiempo y recursos.

Veamos un ejemplo. En primer lugar sustituimos los parámetros por signos de interrogación:

```
String query = "SELECT * FROM Personas WHERE NOMBRE = ? AND APELLIDOS = ?";
```

La consulta se envía al crear el statement, en ese momento es cuando se precompila en el motor de base de datos:

```
PreparedStatement stmt = conn.prepareStatement(query)
```

Una vez preparada la consulta, asignamos los valores a los parámetros (empezando por 1, no por 0):

```
stmt.setString(1, "MARIA");
```

```
stmt.setString(2, "GARCIA PEREZ");
```

Y finalmente se ejecuta. Obsérvese que ahora no se pasan argumentos al método executeQuery:

```
ResultSet rs = stmt.executeQuery();
```

A continuación se muestra el código completo:

```
package tema2;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

import org.sqlite.SQLiteDataSource;

public class PreparedStatementExample {
    public static void main(String[] args) {

        SQLiteDataSource ds = new SQLiteDataSource();
        ds.setUrl("jdbc:sqlite:C:/cursojava/ejemplo1.db");

        String query = "SELECT * FROM Personas WHERE NOMBRE = ? AND APELLIDOS = ?";

        try (Connection conn = ds.getConnection();
            PreparedStatement stmt = conn.prepareStatement(query)); {

            stmt.setString(1, "MARIA");
            stmt.setString(2, "GARCIA PEREZ");

            ResultSet rs = stmt.executeQuery();

            while (rs.next()) {
                int id = rs.getInt("ID");
                String nombre = rs.getString("NOMBRE");
                String apellidos = rs.getString("APELLIDOS");

                System.out.println("Fila: ID = " + id + ", NOMBRE = " + nombre + ",
                    APELLIDOS = " + apellidos);
            }
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
            e.printStackTrace(System.err);
        }
    }
}
```

La ventaja más importante de los objetos PreparedStatement es que ayudan a prevenir ataques de inyección SQL. La inyección SQL es una técnica para explotar maliciosamente las aplicaciones que utilizan datos proporcionados por el cliente en declaraciones SQL. Los atacantes engañan al motor SQL para que ejecute comandos no deseados mediante el suministro de una entrada de cadena especialmente diseñada, con lo que obtienen acceso no autorizado a una base de datos para ver o manipular datos restringidos. Todas las técnicas de inyección de SQL explotan una sola vulnerabilidad en la aplicación: los literales de cadena incorrectamente validados o no validados se concatenan en una declaración SQL construida dinámicamente y el motor SQL los interpreta como código.

Por ejemplo, supongamos que en lugar de emplear PreparedStatement, hubiésemos empleado un Statement cuya sql se construye dinámicamente con la información recibida por parámetro. Algo semejante a esto:

```
package tema2;

import java.sql.Connection;
import java.sql.Statement;
import java.sql.ResultSet;
import java.sql.SQLException;

import org.sqlite.SQLiteDataSource;

public class ExampleSQLInjection{
public static void main(String[] args) {

    SQLiteDataSource ds = new SQLiteDataSource();
    ds.setUrl("jdbc:sqlite:C:/cursojava/ejemplo1.db");

    String query = "SELECT * FROM Personas WHERE NOMBRE =\"" + args[0] + "\"";

    try (Connection conn = ds.getConnection();
        Statement stmt = conn.createStatement();) {

        ResultSet rs = stmt.executeQuery(query);

        while (rs.next()) {
            int id = rs.getInt("ID");
            String nombre = rs.getString("NOMBRE");
            String apellidos = rs.getString("APELLIDOS");

            System.out.println("Fila: ID = " + id + ", NOMBRE = " + nombre +
                ", APELLIDOS = " + apellidos);
        }
    } catch (SQLException e) {
        System.err.println("Error: " + e.getMessage());
        e.printStackTrace(System.err);
    }
}
```

Al ejecutarlo con un parámetro MARIA devolvería el resultado esperado. Pero la Inyección de SQL consisten en que nos pasen por parámetro una sql y que construyamos una sentencia que el atacante desea ejecutar. Por ejemplo, si en este caso recibimos en lugar de MARIA el siguiente parámetro:

```
"MARIA \" UNION SELECT * FROM PERSONAS\""
```

Nuestro programa devolverá todas las personas, sin tener en cuenta el parámetro. Otra opción sería ir jugando para ejecutar sentencias sobre la tabla USUARIOS, PASSWORDS, etc... Tablas que en este caso no existen, pero podrían existir.

Los objetos PreparedStatement siempre tratan los datos proporcionados por el cliente como contenido de un parámetro y nunca como parte de una sentencia SQL, de modo que no es posible realizar la Inyección de SQL.

7. Ejercicio resuelto

Se propone crear una aplicación que lea de un fichero de texto un listado de datos de personas y los inserte en una tabla de la base de datos. El fichero de texto tiene los valores separados por punto y coma.

La tabla tendrá el nombre Clientes.

Los campos de la tabla serán los siguientes:

- ID: numérico (no autogenerado)
- NOMBRE: texto
- APELLIDOS: texto
- FECHA_NACIMIENTO: fecha (en sqlite el tipo fecha no existe como tal, se puede usar el formato texto y almacenar por ejemplo en formato YYYY-MM-DD para hacer comparaciones).
- NUM_PEDIDOS: numérico

Finalmente se consultará la tabla para mostrar un listado de todas las personas nacidas antes del 01/01/1978 cuyo número de pedidos sea superior a 10.

Se proporciona el script sql para crear la base de datos, el fichero de texto con los datos así como la solución al ejercicio, todo ello se puede descargar en la plataforma. Si se hace correctamente el resultado debe ser un listado con 7 personas.

8. Recursos

- Tutorial básico sintaxis SQL:
<https://www.w3schools.com/sql/default.asp>
- Tutorial avanzado JDBC (en inglés):
<https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html>